PONTIFÍCIA UNIVERSIDADE CATÓLICA DE CAMPINAS
Projetos de Sistemas Operacionais
1s/2026 – Prof. Lucas Reis

Laboratório 1 - Hello, World e Chamadas de Sistema

Este documento apresenta um questionário de laboratório prático sobre chamadas de sistema (*syscalls*) dentro de um kernel Linux, afim de familiarizar o aluno com esse coneito fundamental em sistemas operacionais. O objetivo é explorar como os sistemas operacionais aplicam a transição entre modo de usuário e modo de kernel, assim como explorar a interface de programação de aplicação (API) POSIX para chamadas de sistema.

1. Um conjunto de questões estão espalhadas neste questionário. Elas têm o formato a seguir:

Exemplo de enunciado - Qual o nome da disciplina?
Projetos de Sistemas Operacionais

   Elas devem ser respondidas **no próprio documento** por meio do seu editor de PDFs favorito. Uma versão do PDF editada com as respostas deve ser enviada pelo Canvas até o final do dia anterior (23:59) à nossa próxima aula de laboratório.

2. Para facilitar o desenvolvimento, os arquivos necessários para realização desse laboratório estão no Canvas com o nome de `lab1.zip`. Entretanto, eu recomendo fortemente **digitar os arquivos manualmente**. Isso te força a ler o arquivo manualmente linha a linha, auxiliando o entendimento dos programas trabalhados neste laboratório.

3. Esse documento assume que você possui a máquina virtual construída durante o processo do primeiro laboratório (`Lab0`), **incluindo a configuração do diretório compartilhado entre o host e a máquina virtual**. Caso você não tenha feito/refeito esses passos, não será possível realizar este laboratório.

Preparando o Ambiente

Antes de começar, precisamos instalar os pacotes referentes aos binários que usaremos nessa aula (e em várias outras durante o semestre). É sempre prudente manter o sistema atualizado, então vamos iniciar por esse passo:

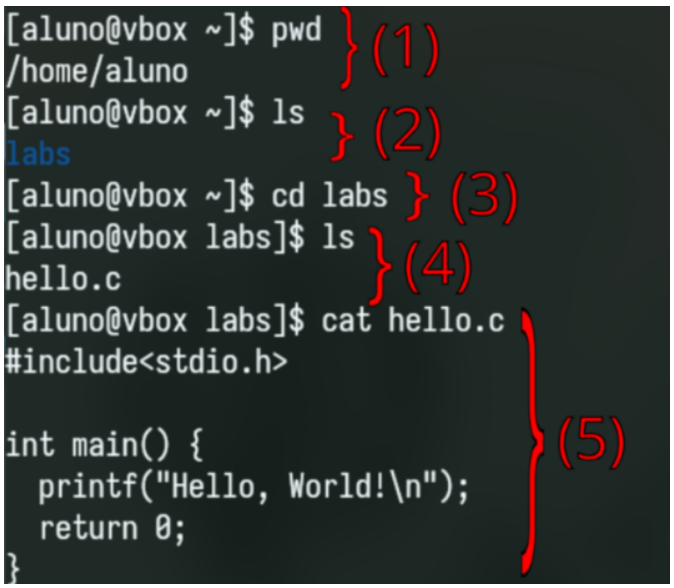
```
sudo dnf -y update
sudo dnf -y install gcc strace
```

1 Hello, World!

Para testar nossa máquina virtual e seus conhecimentos com ferramentas de desenvolvimento pelo emulador de terminal, vamos começar pelo clássico *Hello, World!*. Utilizando o editor de texto de sua preferência, crie um arquivo `hello.c` na raíz da pasta compartilhada no sistema hospedeiro. Neste arquivo, escreva um código em C para imprimir *Hello, World* na tela utilizando a função `printf()`.

Na máquina virtual, vamos navegar até essa pasta.

```
pwd
ls
cd labs
ls
cat hello.c
```



O comando `pwd` (1) mostra o caminho para o diretório atual, `ls` (2 e 4) lista todos os arquivos/pastas dentro do diretório. Com `cd` (change directory) (3), podemos saltar para um novo diretório. Por fim, imprimimos na tela o conteúdo do arquivo hello.c com `cat` (5).

Sempre que tiver dúvidas sobre o que um comando no terminal faz, você pode usar o manual do Linux para dar uma descrição mais detalhada do comando.

```
man cat
```

Qual a utilidade do comando `cat`? Interprete os resultados do comando *man* para entender melhor.

No caso, cat é um comando que está lendo o conteúdo do arquivo hello.c e está printando no terminal.
Com o comando "man cat" foi possível ler que o comando concatena arquivos e os envia como saída, pode ser para outro comando, para a saída padrão - que é o terminal - ou diretamente para um arquivo.
Em suma, isso significa que o comando cat lê um ou mais arquivos e passa seus conteúdos (ou parte deles) como saída.

Para compilar e executar o programa, utilize:

```
gcc hello.c  -o hello
./hello
```

GCC é o nome do compilador (GNU Project C Compiler). O parâmetro `-o hello` serve para indicar o nome do binário executável. Para executar o programa utilizamos `./hello`.

Vamos observar as bibliotecas que foram carregadas pelo linker utilizando o programa `lld`:

```
ldd hello
```

Escreva abaixo a linha de saída do comando `ldd` que indica onde está a biblioteca padrão do C (`libc`)

A linha que indica onde está a biblioteca libc é:
libc.so.6 => /lib64/libc.so.6 (0x00007f563fb97000)

Terminal completo do resultado de "ldd hello":

```
pdesio@vbox:~/labs$ ldd hello
        linux-vdso.so.1 (0x00007f563fd98000)
        libc.so.6 => /lib64/libc.so.6 (0x00007f563fb97000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f563fd9a000)
```

2 Chamadas de Sistema

Vamos testar o uso de algumas chamadas de sistema por meio de programas na linguagem C.

Para o primeiro teste, utilizando seu editor de texto favorito, crie o arquivo `escrita.c` dentro da pasta compartilhada por meio do seu sistema hospedeiro:

```c
#include <stdio.h>
#include <unistd.h>
#include <sys/syscall.h>

int main() {
  const char msg[] = "Essa mensagem ocorreu por meio de uma syscall!\n";

  // write(fd, buf, size)
  long err = syscall(SYS_write, 1, msg, sizeof(msg) - 1);
  if (err == -1) {
    fprintf(stderr, "Erro ao realizar a chamada de sistema!\n");
    return 1;
  }
  return 0;
}
```

Compile e execute o programa:

```
gcc escrita.c -o escrita
./escrita
```

O programa retorna como saída uma única linha de texto com uma cadeia de caracteres. Agora observe atentamente o código do programa. Você verá que, apesar do que estamos acostumados a ver quando escrevemos um programa em C, não há uso da função `printf` aqui. No lugar disso, há apenas uma função `syscall` que realiza o trabalho de escrever a mensagem na tela.

Para entender melhor como funciona a função syscall, podemos novamente recorrer ao manual:

```
man syscall
```

Explique como funciona a função `syscall`. Quais são seus argumentos esperados?

Syscall são a maneira de aplicações, preocessos/threads (espaço do usuário) rodarem comandos que precisão de permissão de kernel. Normalmente não utilizamos diretalmente, mas sim por meio de funções wrapper (são funções que encapsulam as syscalls de forma sua utilização mais amigavel). Argumentos:
long syscall(long number, ...);
- number: o número identificador da syscall que você quer chamar
- ...: os argumentos específicos daquela syscall

Neste exemplo, chamamos indiretamente a syscall `write`. Essa chamada de sistema escreve em um descritor de arquivo, um objeto responsável por apontar para um

arquivo aberto pelo processo. Para saber mais sobre uma chamada de sistema específica, podemos usar o manual. Podemos passar como parâmetro um número para especificar que queremos checar uma seção específica do manual. A seção 2 contêm o manual de todas as chamadas de sistema:

```
man 2 write
```

Para mais informações sobre as seções disponíveis, você pode verificar a página do próprio manual:

```
man man
```

Apesar da descrição, a chamada de sistema `write` está escrevendo na tela, e não em um arquivo. Por que?

Acredito que isso aconteça de forma similar ao cat, onde o standart output é o destino do write caso um arquivo não seja especificado. Então o standart output do write deve ser também o terminal.

Podemos utilizar o `strace`, um binário que imprime na tela as chamadas de sistema feitas por um programa. Leia atentamente a descrição do comando:

```
man strace
```

Após ler a descrição, utilize o `strace` para obter a lista de chamadas de sistema que são feitas pelo programa `escrita`.

Escreva o comando completo que você digitou para utilizar o `strace` no programa `escrita`.

strace ./escrita

Escreva a linha onde acontece a chamada de sistema `write`, obtida por meio da chamada do `strace`

write(1, "Essa mensagem ocorreu por meio d"..., 47) = 47

Agora, vamos modificar esse código. Observe novamente a linha 9 do programa `escrita.c` e repare no primeiro argumento passado para a função. Modificando este argumento, altere o valor `SYS_write` para um valor numérico maior que 500. Compile novamente o programa e execute.

O que aconteceu? Teste a execução com `strace` novamente para obter uma resposta mais detalhada.

Pela linha do strace:
syscall_0x1f5(0x1, ...) = -1 ENOSYS (Função não implementada)
Acredito que o argumento 501, ou seja, chamar a syscall 501, não funciona porque não é uma syscall que existe realmente.
(501 foi o valor que eu coloquei no lugar de SYS_write)
No lugar é printado a mensagem erro que foi implementada no código:
Erro ao realizar a chamada de sistema!

Para entender melhor o número de cada syscall, você pode observar o arquivo `/usr/include/asm/unistd_64.h`

Escolha uma chamada de sistema específica observando o arquivo acima. Responda qual o número dela e sua funcionalidade. Lembre-se de usar o manual `man`

A chamada de sistema escolhida foi tkill, como o nome sugere se trata de uma syscall de target kill para thread. No manual esta escrito que é uma syscall de uso para outras bibliotecas thread e que por sinalizar qual thread matar especificamente, baseado no thread ID, não é recomendado que seja utilizado pelo usuário, pode acabar resultando na thread errada sendo sinalizada.

Mapeando Chamadas de Sistema

Agora, vamos observar melhor como funciona o fluxo de chamadas de sistema em um programa comum. Usando seu editor de texto favorito, escreva o programa abaixo na pasta compartilhada com o nome de `arquivo.c`:

```c
#include <stdio.h>
#include <stdlib.h>

/*
 * Este programa le uma string como primeiro argumento e
 * a escreve no arquivo passado no segundo argumento!
 */

int main(int argc, char **argv) {
  if (argc < 3) {
    fprintf(stderr, "Uso: %s string_entrada arquivo_saida", argv[0]);
    return 1;
  }

  FILE *f = fopen(argv[2], "w");

  if (f == NULL) {
    fprintf(stderr, "Nao consegui abrir o arquivo de saida!\n");
    return 1;
```

```
  }

  fprintf(f, "%s\n", argv[1]);
  fclose(f);
  return 0;
}
```

Este programa lê uma string como primeiro argumento e a escreve no arquivo passado no segundo argumento. Compile esse programa em um binário chamado `arquivo`. Uma execução comum do programa pode ser feita da seguinte maneira:

```
./arquivo "serei escrito no arquivo teste" teste.txt
```

Agora, você deve catalogar as chamadas de sistema desse programa utilizando strace. Queremos catalogar as chamadas para `open`,`openat`,`write` e `close` para averiguar quando ele abre e escreve no arquivo especificado nos parâmetros.

Utilizando o strace, catalogue as chamadas de sistema relacionadas à abertura, escrita e fechamento do arquivo especificado pelo uso do programa. Escreva abaixo **apenas as syscalls** utilizadas para fazer a manipulação do arquivo.

```
openat(AT_FDCWD, "hello.c", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
write(3, "teste\n", 6)              = 6
close(3)                           = 0

(eu não criei um novo arquivo para testar o executavel arquivo,
no lugar usei o arquivo hello.c como segundo argumento).
```

Dica: você pode usar a flag `-e` para pedir ao strace catalogar apenas as chamadas de sistema passadas nela. Você também pode usar a flag `-o` para redirecionar a saída do `strace` para um arquivo. Por exemplo:

```
strace -e fork,open,close -o saida_strace.txt ./programa_qualquer
```

Neste exemplo, pedimos ao `strace` que imprima apenas as chamadas para as syscalls `fork`, `open` e `close` no programa `programa_qualquer`, redirecionando a saída para o arquivo `saida_strace.txt`.

O custo de uma chamada de sistema

Como vimos em sala de aula, chamadas de sistema não são gratuitas. A troca de contexto entre o espaço de usuário e o espaço de kernel tem um custo que, apesar de baixo, é perceptível dada a quantidade de chamadas de sistema feitas por um processo qualquer. Para exemplificar, considere o seguinte código:

```
#include <stdio.h>
#include <sys/syscall.h>
#include <sys/time.h>
#include <unistd.h>

#define ITERACOES 1000000

int main(int argc, char *argv[]) {
```

```
  if (argc != 2) {
    fprintf(stderr, "Uso: %s <l|s>\n"
                    "\tl => libc (vDSO)\n"
                    "\ts => syscall (Raw)\n",
            argv[0]);
    return 1;
  }

  struct timeval tv;
  char opcao = argv[1][0];

  if (opcao == 'l') {
    for (int i = 0; i < ITERACOES; i++)
        gettimeofday(&tv, NULL);

    printf("Glibc (vDSO): %ld sec, %ld usec (Ultima iteracao)\n", tv.
       tv_sec, tv.tv_usec);
  } else if (opcao == 's') {
    for (int i = 0; i < ITERACOES; i++)
        syscall(SYS_gettimeofday, &tv, NULL);

    printf("Syscall (Raw): %ld sec, %ld usec (Ultima iteracao)\n", tv
       .tv_sec, tv.tv_usec);
  } else {
    fprintf(stderr, "Opcao invalida!\n");
    return 1;
  }

  return 0;
}
```

Este programa é usado para obter quantos segundos se passaram desde o *Epoch* (1/1/1970 - 00:00) por meio da chamada de sistema `gettimeofday`. Para medir o desempenho das chamadas de sistema, o programa faz essa chamada milhares de vezes. Esse programa também conta com duas opções de parâmetros (`s` e `l`).

Compile o programa em um binário chamado `horario` e teste seu uso com ambas as opções:

```
./horario l # usa libc/vdso
./horario s # faz a syscall manualmente
```

A primeira opção usa a função padrão da libc que encapsula a chamada `gettimeofday`. O motivo para esse encapsulamento está em utilizar, quando possível, o `vDSO`, uma biblioteca virtual que mapeia essas chamadas de sistema para o espaço de usuário, evitando assim o custo da troca de contexto entre o modo de usuário e modo de kernel. Você pode ler mais sobre isso na página do manual:

```
man vdso
```

Apesar da transição entre espaço de usuário e espaço de kernel exsitirem para garantir também a segurança do controle dos recursos do computador pelo kernel, algumas chamadas de sistema são inofensivas e não precisam desse controle. Por isso

essa solução, que pode mapear outras chamadas de sistema como `getcpu`. As chamadas de sistema mapeadas para a `vDSO` para cada arquitetura de processador estão na página do manual citada acima.

Você pode confirmar que essa biblioteca é mapeada para um programa qualquer utilizando a ferramenta `ldd`.

Utilizando o `ldd` em um binário qualquer, encontre a linha que diz onde é carregada a biblioteca `vDSO`. Escreva essa linha abaixo.

linux-vdso.so.1 (0x00007f7ab4c1d000)

Agora que entendemos a existência dessa biblioteca virtual e quais seus benefícios, vamos medir o ganho de desempenho obtido com ela. Para isso, podemos utilizar novamente o `strace`. Esse comando possui uma flag `-c`, que computa métricas de tempo gasto em todas as chamadas de sistema feitos no seu programa.

Utilizando o `strace` com o parâmetro `-c`, verifique quanto tempo é gasto a mais quando executamos forçamos as chamadas de sistema no programa `horario` (parâmetro `s`) em comparação a usando a `vDSO` (parâmetro `l`). Escreva abaixo o **tempo total** gasto em chamadas de sistema e o **tempo gasto em chamadas para `gettimeofday`** em ambas as execuções.

parâmetro s:
99,99 1,267597 1 1000000 gettimeofday
100,00 1,267671 1 1000033 1 total
parâmetro l:
100,00 0,000000 0 33 1 total
(nem achei o gettimeofday)

A diferença foi 1,27 - 0 = 1,27